Floating point and short-circuit evaluation

Numbers in binary, why `0.1 + 0.2` is not 0.3, how to compare floats, and why `and` and `or` can skip a side.

THE QUESTION

Why is `0.1 + 0.2 == 0.3` False, and why does `x != 0 and 10 / x > 1` never divide by 0?

THE INTUITION

A float has a fixed number of binary digits, so 0.1 is kept as the nearest binary fraction, a tiny bit off. `and` and `or` stop once the answer is known.

KEY TAKEAWAY

Compare floats with `math.isclose`, never with `==`. Put the safety test on the left of `and`.

you type 0.1 → stored in binary → cut to 53 bits → a tiny error → errors add up → compare with a tolerance

1 Binary: every place is a power of 2

Decimal: places are powers of 10

10	1	
1	3	1 × 10 + 3 × 1 = 13

Binary: places are powers of 2

8	4	2	1	
1	1	0	1	8 + 4 + 0 + 1 = 13

Decimal to binary: divide by 2, keep the remainder

```
13 // 2 = 6    13 % 2 = 1
 6 // 2 = 3     6 % 2 = 0
 3 // 2 = 1     3 % 2 = 1
 1 // 2 = 0     1 % 2 = 1
```

read the remainders bottom up: 1101

```
>>> bin(13)
'0b1101'
>>> int("1101", 2)
13
```

In **decimal** (base 10) each place is worth 10 times the one to its right. Computers use **binary** (base 2): only the digits 0 and 1, called **bits**, in places worth 1, 2, 4, 8 and so on. To convert, divide by 2 with `//` and `%` (see the variables sheet) until you reach 0. `bin()` gives the binary text; `int(text, 2)` goes back.

2 After the point: halves, quarters, eighths

0.625: reaches 0, so it ends

fraction × 2	bit
0.625 × 2 = 1.25	1
0.25 × 2 = 0.5	0
0.5 × 2 = 1	1

0 left: stop

0.625 = 0.101 in binary

1	0	1	
1/2	1/4	1/8	1/2 + 1/8 = 0.625

0.1: back to 0.2, so it repeats

fraction × 2	bit
0.1 × 2 = 0.2	0
0.2 × 2 = 0.4	0
0.4 × 2 = 0.8	0
0.8 × 2 = 1.6	1
0.6 × 2 = 1.2	1
0.2 × 2 = 0.4	0

row 6 repeats row 2, and so on for ever

0.1 = 0.0 0011 0011 0011 ...

Binary places after the point are worth 1/2, 1/4, 1/8 and so on. To convert a fraction, double it: the whole part is the next bit; keep the rest and repeat. 0.1 comes back to 0.2, so its bits repeat for ever, as 1/3 = 0.333… does in decimal.

3 53 bits: 0.1 is stored slightly off

0.1 cut off after a few bits: close, never equal

4 bits	0.0001	= 0.0625
8 bits	0.00011001	= 0.09765625
16 bits	0.0001100110011001	= 0.0999908447265625

A float keeps 53 bits, from the first 1, and rounds the rest:

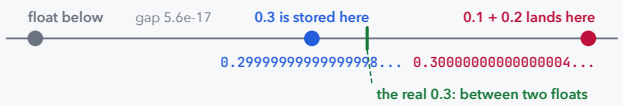
```
>>> print(0.1)  0.1  the shortest text for this float
>>> f"{0.1:.20f}"  '0.10000000000000000555'  20 places
>>> print(Decimal(0.1))
0.1000000000000000055511151231257827021181583404541015625
>>> (0.1).hex()  '0x1.999999999999ap-4'
```

A **float** keeps 53 bits, so 0.1 becomes the nearest binary fraction: a **rounding error** of about 5.6×10^{-18}. `print` shows the shortest text that leads back to the same float, which hides it. `Decimal` (from the `decimal` module) shows the exact stored value; `.hex()` shows the bits in base 16 (9 is 1001; `p-4` means times 2^{-4}).

4 `0.1 + 0.2` lands one float above 0.3

```
>>> 0.1 + 0.2                                  0.30000000000000004
>>> 0.1 + 0.2 == 0.3                           False
>>> 0.1 + 0.1 == 0.2                           True
>>> math.nextafter(0.3, 1) == 0.1 + 0.2        True
```

Zoom in on 0.3: neighbouring floats, one gap apart



0.1 and 0.2 are both stored a little high; their sum rounds to the float just above the one holding 0.3. `==` compares stored values exactly, so the notebook's `if 0.1 + 0.2 == 0.3:` prints "not equal". `math.nextafter(0.3, 1)`, the next float after 0.3, is the sum. Some sums do come out right.

5 Small errors pile up as you keep adding

total = 0.0, then total += 0.1 again and again

k	total (printed)	k / 10	same?	k	total (printed)	k / 10	same?
0	0.0	0.0	True	6	0.6	0.6	True
1	0.1	0.1	True	7	0.7	0.7	True
2	0.2	0.2	True	8	0.7999999999999999	0.8	False
3	0.30000000000000004	0.3	False	9	0.8999999999999999	0.9	False
4	0.4	0.4	True	10	0.9999999999999999	1.0	False
5	0.5	0.5	True				

k counts the additions · rose: 4 of 11 differ from k / 10

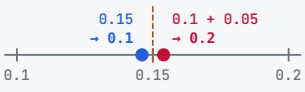
Each `+=` rounds to the nearest float, so errors can build up: after ten additions `total` prints `0.9999999999999999` and `total == 1.0` is `False`. A loop repeats a line for you (see the while loops sheet). For money, count whole paise or cents as ints, which are exact.

6 Compare floats with a tolerance, not ==

```
>>> a = 0.1 + 0.2
>>> a == 0.3                                   False
>>> abs(a - 0.3) < 0.0001                      True
>>> math.isclose(a, 0.3)                       True
>>> math.isclose(a - 0.3, 0)                   False
>>> math.isclose(a - 0.3, 0, abs_tol=1e-9)     True
```

round() first? Nearly equal floats can round apart:

```
b = 0.1 + 0.05
round(b, 1) == round(0.15, 1)  False
math.isclose(b, 0.15)          True
```



A **tolerance** says how close counts as equal; the notebook tests `abs(actual - expected) < tolerance` with 0.0001. `math.isclose(a, b)` (after `import math`) measures closeness relative to the size of the numbers; near 0, pass `abs_tol`. Rounding both sides first does not help.

Exact ints, float limits, and conditions that stop early

Where floats run out, why and and or can skip their right side, and which value they hand back.

7 An int is exact; a float has limits

you type	Python gives
`2 ** 100`	`1267650600228229401496703205376`
`10**20 + 1 - 10**20`	`1`
`1e20 + 1 - 1e20`	`0.0`
`sys.float_info.max`	`1.7976931348623157e+308`
`sys.float_info.max * 2`	`inf`
`2.0 ** 1024`	`OverflowError: (34, 'Result too large')`
`sys.float_info.epsilon`	`2.220446049250313e-16`
`1 + 1e-17 == 1`	`True`

int: exact, any number of digits · float: about 16 digits, a limited range

An **int** has no rounding error and no size limit (`10 ** 400` has 401 digits). A float keeps about 16 significant digits, so `1e20 + 1` loses the 1 (`1e20` is **scientific notation** for 10^{20}). `sys.float_info` (after `import sys`) lists the limits: past the max comes `inf` (infinity) or an **OverflowError**. **Epsilon** is the gap from 1.0 to the next float.

8 and and or stop once the answer is known

```
def check(name, answer):
    print("checking", name)
    return answer
```

check prints its name, then gives back its answer
struck out: never called

you type	printed	value
`check("A", False) and ~~check("B", True)~~`	`checking A`	`False`
`check("A", True) and check("B", False)`	`checking A` `checking B`	`False`
`check("A", True) or ~~check("B", False)~~`	`checking A`	`True`
`check("A", False) or check("B", True)`	`checking A` `checking B`	`True`

and stops at the first False · or stops at the first True

Short-circuit evaluation: Python works out the left side first. If the left of and is False, so is the answer, and the right side is never **evaluated** (worked out); for or, a True left is enough. `check` is a small **function** (see the functions sheet) that prints when it is called.

9 Put the safety test first

guard first, with x = 0

```
x != 0 and 10 / x > 1
```

1. x != 0 gives False
2. and stops: 10 / x never runs

```
prints: skip
```

guard second, with x = 0

```
10 / x > 1 and x != 0
```

1. 10 / x runs first
2. dividing by 0 stops the program

```
ZeroDivisionError:
division by zero
```

with x = 5 both orders print big: the guard only matters when x is 0

A **guard** is a test that makes the next one safe. On the left of and, `x != 0` stops Python before `10 / x` when x is 0; swapped, the division runs and fails with a **ZeroDivisionError**. The notebook's `0==1 and 1/0==1` prints No, but `0==1 or 1/0==1` crashes: or needs its right side when the left is False.

10 and and or hand back one of their operands

you type	value	why
`3 and 7`	`7`	3 counts as true: and gives 7
`0 and 7`	`0`	0 counts as false: and stops
`0 or 5`	`5`	0 counts as false: or gives 5
`"" or "guest"`	`'guest'`	empty text counts as false
`"Bob" or "guest"`	`'Bob'`	'Bob' counts as true: or stops
`bool(0), bool("")`	`(False, False)`	zero, empty text: false
`bool(-2), bool("0")`	`(True, True)`	other numbers and text: true

a and b: if a counts as false, gives a, else b
a or b: if a counts as true, gives a, else b

An **operand** is a value an operator works on; and and or give back one of theirs, not always True or False. 0, 0.0 and empty text `""` **count as** false, other numbers and text as true (see the nested conditions sheet). So `typed or "guest"` gives `'guest'` when `typed` is `""`.

11 Common mistakes: what you wrote, what Python says or prints, the fix

you wrote (x is 0)	the last line Python prints	why, and the fix
`0.1 + 0.2 == 0.3`	no error; it gives False	use `math.isclose(0.1 + 0.2, 0.3)`: True
`math.isclose(0.1, 0.1)`	`NameError: name 'math' is not defined`	write `import math` first
`10 / x > 1 and x != 0`	`ZeroDivisionError: division by zero`	guard first: `x != 0 and 10 / x > 1`
`x != 0 or 10 / x > 1`	`ZeroDivisionError: division by zero`	left False, so or runs the right; use and
`math.isclose(1e-20, 0)`	no error; it gives False	near 0 add `abs_tol=1e-9`: then True
`2.0 ** 1024`	`OverflowError: (34, 'Result too large')`	past the float max; the int `2 ** 1024` works

12 Try it (answers in green)

1. `print(0.1 + 0.1 + 0.1 == 0.3)`?
`False`, as the sum is not 0.3.

2. Write 11 in binary.
1011, as 8 + 2 + 1; `bin(11)` gives `0b1011`.

3. What is binary 0.11 in decimal?
1/2 + 1/4: `0.5 + 0.25` gives `0.75`.

4. `print(0 and 1/0, 1 or 1/0)`?
`0 1`: neither division runs.

5. `print("" or "none", 4 and 0)`?
`none 0`: `""` is false, 4 is true.

6. With `x = 0`, is `10 / x < 2 or x == 0` safe?
No: a ZeroDivisionError.
`x == 0 or 10 / x < 2` gives `True`.

floats are binary approximations · compare them with `math.isclose` · ints are exact · and, or stop early: guard first

Source: the floating point and conditionals notebook (its floating-point and short-circuit part) and the handwritten notes on binary numbers and floats, nipunbatra.github.io/comp22. Every output and error message was produced by running the code with Python 3.11.11; other versions word some messages differently. Next: the nested and chained conditions sheet.